

Fatigue Routine v7_2

How to Add a New Method (Step-by-step)

2026-02-25

1 What “adding a method” means in this codebase

The repository supports two kinds of fatigue methods:

1. **Critical Plane (CP) methods** (plane-level plugins)
2. **Direct / invariant methods** (case-level models)

1.1 1) Critical Plane (CP) methods

A CP method is implemented as a *plane-level* class in:

`fatigue/models/cp_methods/`

It receives the stress/strain tensors *already rotated to a candidate plane* and returns two scalars:

- **damage**: the damage parameter on that plane.
- **metric**: a selection metric used to choose the “critical” plane.

The case-level wrapper `fatigue/models/cp_adapter.py` evaluates the method on all sampled planes and returns:

`values[""]` = damage on the plane that maximizes `metric`
`values["_ext"]` = maximum damage over all planes

Out-of-phase loading. Phase shift handling is applied *centrally* in `fatigue/eval/evaluator.py` using harmonic coefficients from the Excel file. So most CP methods do not need any special logic for out-of-phase loading.

1.2 2) Direct / invariant methods

A direct method is a *case-level* model implemented in one of these folders:

`fatigue/models/invariant_methods/`
`fatigue/models/energy_based_methods/`

It receives the full case tensors and must return a `CaseResult` containing one or more scalar outputs in a dictionary.

2 Before you start: how names, families, and styles work

2.1 Method name

The user-facing name of a method is the class attribute `name` (examples: FS, FIN, OTT, ZHU_EDP).

2.2 “_ext” extended variant

For CP methods, `_ext` is always available via the CP adapter. In plots and in the summary table, `base` and `_ext` share the same color and marker:

- `base` method: solid line, filled marker
- `_ext`: densely dashed line, marker with `fill=white`

2.3 Method family

Each CSV row contains `Method_family`, inferred from the Python module path. This is used by `2_compare_results.py` to build a per-family color palette (e.g. CP methods in a “red” palette, invariant methods in a “blue” palette).

3 A. Adding a new Critical Plane (CP) method

3.1 Step A1: create the method file

Create a new file in:

`fatigue/models/cp_methods/`

Example: `fatigue/models/cp_methods/mycp.py`

Use this template:

```
from __future__ import annotations
from typing import Mapping
import numpy as np

from fatigue.models.base import CPPlaneResult

class MyCP:
    """Example critical-plane method."""

    name = "MYCP" # user-facing name

    def required_params(self) -> set[str]:
        # List ALL material parameters your formula needs.
        return {"Sy"} # example

    def evaluate_on_plane(
        self,
        *,
        S0r: np.ndarray,
        E0r: np.ndarray,
        S1r: np.ndarray,
        E1r: np.ndarray,
        params: Mapping[str, float],
    ) -> CPPlaneResult:
        """Compute damage + metric on ONE plane.

        Inputs are already rotated into the candidate plane frame.
        Convention in this repository:
        - plane normal is local z' (index 2)
        - in-plane shear components relevant to CP are xz and yz
        """

        # --- example: compute a scalar from a shear range ---
        dtau_xz = S0r[0, 2] - S1r[0, 2]
        dtau_yz = S0r[1, 2] - S1r[1, 2]
```

```

dtau = float(np.sqrt(dtau_xz**2 + dtau_yz**2))

Sy = float(params["Sy"])

damage = dtau / Sy
metric = damage # often metric == damage, but can differ
return CPPlaneResult(damage=float(damage), metric=float(metric))

```

3.2 Step A2: register the CP method

Open:

fatigue/models/registry.py

Add:

- an import line near the other CP imports
- an entry in `_available_cp_methods()` mapping "MYCP": `MyCP()`.

3.3 Step A3: run it (calibration + evaluation)

1. Add "MYCP" to `MODEL_NAMES` in `1_run_material.py` (so it is calibrated on the design subset).
2. Set `MODEL_NAME = "MYCP"` or `MODEL_NAME = "MYCP_ext"`.
3. Run:

```
python 1_run_material.py
```

3.4 Step A4: validate quickly

A new CP method is “healthy” if:

- it returns *finite, positive* values for the uniaxial design subset (otherwise the fit fails);
- `required_params()` are all provided (otherwise `1_run_material.py` stops with a clear error);
- it behaves for both in-phase and out-of-phase cases (phase handling is done in the evaluator).

4 B. Adding a new direct (case-level) model

4.1 Step B1: choose the family folder

Pick one of:

- `fatigue/models/invariant_methods/` for stress/strain invariants,
- `fatigue/models/energy_based_methods/` for energy-based formulas,
- or create a new family folder if needed (see Section 6).

4.2 Step B2: implement the model

Example file: `fatigue/models/invariant_methods/myinv.py`

```

from __future__ import annotations
from typing import Mapping
import numpy as np

from fatigue.models.base import CaseResult

```

```

class MyInvariant:
    name = "MYINV"

    def required_params(self) -> set[str]:
        return {"Su"} # example

    def evaluate_case(
        self,
        *,
        S0: np.ndarray,
        E0: np.ndarray,
        S1: np.ndarray,
        E1: np.ndarray,
        params: Mapping[str, float],
        R_list=None,
        harmonics=None,
    ) -> CaseResult:
        # Example: von-Mises stress amplitude from max/min tensors
        dS = 0.5 * (S0 - S1)
        s_dev = dS - np.trace(dS) / 3.0 * np.eye(3)
        vm = float(np.sqrt(1.5 * np.sum(s_dev * s_dev)))
        return CaseResult(values={"": vm})

```

Using phase information in a direct model (optional). If you need non-proportional metrics, use `harmonics` (contains `S_mean`, `S_sin`, `S_cos`, etc.). CP methods get phase support automatically; direct models must use it explicitly.

4.3 Step B3: register the direct model

In `fatigue/models/registry.py`, import your class and add it to `_available_direct_models()`.

4.4 Step B4: run it

Add "MYINV" to `MODEL_NAMES` in `1_run_material.py`, select it via `MODEL_NAME`, and run the script.

5 C. If your method needs new material parameters

Material properties are read from the Excel **Summary** sheet. If you add a new parameter:

1. Add the column to the **Summary** sheet and fill the value in the row referenced by the named cell **Average**.
2. Update the importer mapping in `fatigue/imp/experimental_data.py` (function that searches headers) so your new column is read.
3. In `1_run_material.py`, add the parameter to the `params={...}` dictionary.
4. Add the key to `required_params()` of your method.

6 D. Adding a new method family (rare)

If you create a new folder under `fatigue/models/`, the family name will automatically appear in the CSV as `Method_family`. However, for nicer plots you should also map the family to a color palette.

Open `2_compare_results.py` and edit the helper:
`_family_cmap_name(family)`
so that your new family uses an appropriate sequential colormap (e.g. `Purples`, `Oranges`, ...).

7 E. Troubleshooting checklist (most common)

- **“Missing required material params”**: you forgot to add the parameter to `params` in `1_run_material.py` or to the Excel Summary sheet.
- **Fit fails / returns NaN**: your method returns non-positive or NaN values on the design subset.
- **Method not found**: you forgot to add it to `fatigue/models/registry.py`.
- **You still see old behavior**: delete old CSVs in `Results/` and re-run, or restart your Python kernel (import caching).